

Week 10 Problem Set

Randomised Algorithms

Congratulations on reaching the end of this course!

Please note:

- Just 3+1 exercises this week and only one program to submit.
- **Note the early submission deadline: Friday, 22 November, at 11:00:00am.**

1. (Random numbers)

a. A program that simulates the tosses of a coin is as follows:

```
#define NTOSSES 20
srand(time(NULL));
int i, count = 0;
for (i=0; i<NTOSSES; i++) {
    int toss = rand() % 2;    // toss = 0 or 1
    if (toss == 0) {
        putchar('H');
        count++;
    } else {
        putchar('T');
    }
}
printf("\n%d heads, %d tails\n", count, NTOSSES-count);
```

Sample output is:

```
HHTTTTHHTTTTHHTHHHHHH
12 heads, 8 tails
```

1. What would an analogous program to simulate the repeated rolling of a die look like?
 2. What could be a sample output of this program?
- b. If you were given a string, say "hippopotamus", and you had to select a random letter, how would you do this?
 - c. If you have to pick a random number between 2 numbers, say i and j (inclusive), how would you do this? (Assume $i < j$.)
 - d. Write a C program that generates a random word (consisting of just lower-case letters a–z). When executed, the first command-line argument is the length of the word, and the second argument is the seed used by the random number generator, i.e. by `rand()`.

An example of the program executing could be

```
prompt$ ./randword 6 2
ebgnha
```

which seeds the random-number generator with 2 and generates a random word of length 6.

We have created a script that can automatically test your program. To run this test you can execute the `dryrun` program that corresponds to this exercise. It expects to find a file named `randword.c` in the current directory.

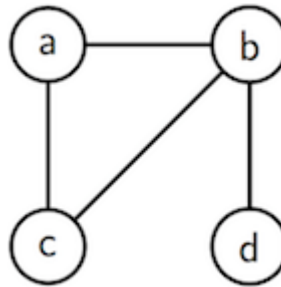
You can use dryrun as follows:

```
prompt$ 9024 dryrun randword
```

Note: It is important in this exercise that you call the random number generator as specified in the lecture, otherwise the outputs will not match. If you use a different method to generate random numbers, the testcases will 'fail', but your program could still be perfectly correct. The output may also be different on your own computer.

2. (Karger's algorithm)

Consider the graph G :



- Find a minimum cut of G .
- Show two different ways in which randomised edge contraction may execute on G , one that results in a minimum cut and one that doesn't.
- Determine the total number of possible executions of the edge contraction algorithm on G . How many of these result in a minimum cut? What, therefore, is the probability for Karger's algorithm to find a minimum cut for G ?

3. (Analysis of randomised algorithms)

Random permutations can be a good way to test the average runtime behaviour of implementations, for example to test sorting algorithms, BST insertion strategies etc.

The following algorithm can be used to compute a uniform random permutation of a given array:

```

randomiseInPlace(A):
|   Input   array A[1..n]
|   Output random permutation of A
|
|   for all i=1..n do
|       swap A[i] with A[random number between i and n]
|   end for
|   return A

```

Does the following variation also produce each possible permutation with equal probability?

```

randomiseWithAll(A):
|   Input   array A[1..n]
|   Output random permutation of A
|
|   for all i=1..n do
|       swap A[i] with A[random number between 1 and n]
|   end for
|   return A

```

4. Challenge Exercise

In cryptography, a *substitution cipher* encrypts a plaintext by replacing each letter with another letter according to a fixed system. For example, if $e \rightarrow y$, $h \rightarrow r$, $s \rightarrow h$, then the plaintext

she sees, he sees.

would be encrypted by this ciphertext:

hry hyyh, ry hyyh.

Even though there are $26! \approx 4 \cdot 10^{26}$ different encryption keys, substitution ciphers are not very strong and cryptanalysts can break them by guessing the meaning of the most frequent letters in a ciphertext. With the help of a [letter-frequency table for English](#), try to decrypt the ciphertext below that's been encrypted using a randomly generated substitution cipher.

fhb fny jnvj: nmfbx fhxbb rnvj pcfhyef ixyqxnookq, zcmb wbyobj
obnkckqzbjj! (qbymmxbv dnobj)

(The first student to email [me](#) the correct plaintext will receive accolades on this webpage.)

Assessment

Group submissions will not be allowed. Your program must be entirely your own work. Plagiarism detection software will be used to compare all submissions pairwise.

Submit your solution to **Exercise 1d** using the following **give** command:

```
prompt$ give cs9024 week10 randword.c
```

Make sure you spell the filenames correctly. You can run **give** multiple times. Only your last submission will be marked.

The deadline for submission is **Friday, 22 November 11:00:00am**.

The program is worth 2 marks. Auto-marking will be run by the lecturer several days after the submission deadline using different test cases than `dryrun` does.

Hints:

- Programs will not be manually marked.
- It is important that the output of your program follows exactly the format shown above, otherwise auto-marking will result in 0 marks.
- Ensure that your program compiles on a CSE machine with the standard options `-Wall -Werror -std=c11`. Programs that do not compile will receive 0 marks.
- `dryrun` and auto-marking also check the correct usage of dynamic memory allocation and pointers as well as the absence of memory leaks.
- Do your own testing in addition to running `dryrun`.